Deep Learning Catch-Up

Weeks 1–8

Part A: Readings, Objectives, Coding Assignments & Practice Questions

Ten tasks take you from Week 1 through Week 8. Each task contains (1) prerequisites to understanding, (2) textbook readings with learning objectives, (3) a practice coding assignment, (4) practice problems of the kind that could appear on a test (general understanding, math needed, application), and (5) broader context. This file contains **no answers**; worked solutions and coding checks are in the separate Part B answer key.

Textbooks used

- **Bishop (PRML):** C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- **DLwPT:** E. Stevens, L. Antiga, T. Viehmann, *Deep Learning with PyTorch*, Manning, 2020.
- **Course materials:** lecture Slide Decks 05–07 (Weeks 6–8).

A note on page numbers

Page ranges are approximate and may differ between printings. Treat the chapter and section titles as the reliable guide and confirm against your copy. DLwPT was written before Transformers and has no RNN or attention chapter, so Tasks 8–10 lean on the slides and the PyTorch documentation for those topics.

How to use this packet

Read the assigned sections, complete the coding assignment, then attempt problems (a)–(c) closed-book before opening Part B.

Progress tracker

Task	Core subject	Primary reading	PyTorch focus
1	ML Foundations & 4-Tuples	Bishop §1.1	Tensors & shapes
2	Linear Regression & Normal Equations	Bishop §1.1, §3.1.1	`torch.linalg`
3	Probabilistic MLE & Gaussian NLL	Bishop §1.2.5	Custom loss functions
4	Classification, Softmax & Cross-Entropy	Bishop §4.2, §4.3.4, §1.6	`nn.CrossEntropyLoss`
5	MLPs & Activations	Bishop §5.1	`nn.Module`, `nn.Sequential`
6	Backprop, Xavier Init, Regularization	Bishop §5.3, §5.5	Autograd, weight decay, dropout
7	CNN Basics: Locality & Weight Sharing	Bishop §5.5.6	`nn.Conv2d`, `nn.MaxPool2d`
8	Modern CNNs, ResNets, 1×1 Convs	Lecture 05 slides	Residual blocks
9	Sequential Modeling & RNNs	Bishop §13.1, Lecture 06	Recurrent loops
10	Attention, Transformers, BERT	Lecture 07 slides	Scaled dot-product attention

Task 1: ML Foundations & Problem Framing

Covers Week 1 material

1. Prerequisites to Understanding

- Basic multivariable algebra and linear notation.
- Python data types and basic `torch.Tensor` structures.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** Introduction and §1.1 (approx. pp. 1–4).
  Objective: Distinguish supervised, unsupervised, and sequential learning paradigms; define representation learning versus hand-crafted feature engineering.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 1 (approx. pp. 3–14) and Chapter 3 (approx. pp. 45–60).
  Objective: Construct and manipulate core PyTorch tensors: `torch.tensor`, shapes, data types, device allocation.

3. Practice Coding Assignment

Create synthetic dataset tensors in PyTorch representing:

1. **Supervised data:** a feature matrix $X \in \mathbb{R}^{100\times 5}$ paired with continuous targets $y \in \mathbb{R}^{100}$.
2. **Unsupervised data:** an unlabeled data matrix $X \in \mathbb{R}^{100\times 5}$.
3. **Sequential data:** a batch of sequences $X \in \mathbb{R}^{32\times 10\times 5}$ (batch size $\times$ sequence length $\times$ feature dimension).

Print each tensor's `shape`, `dtype`, and `device`.

Reading for this assignment: DLwPT Ch. 3: "Tensors: multidimensional arrays", "Tensor element types", "Managing a tensor's device attribute". DLwPT Ch. 4: "Working with time series" (the book shapes sequences as $N \times C \times L$; this assignment uses the batch-first $N \times L \times C$ layout).

4. Practice Test Problems

(a) General Understanding. Define the core **4-tuple** that characterizes any machine learning problem. Explain why deep learning avoids manual feature extraction.

(b) Math Needed. Given a dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N}$ with $x_n \in \mathbb{R}^d$ and $y_n \in \mathbb{R}$, write the scalar expression for the mean squared error objective $\mathcal{L}(\mathbf{w}, b)$ under the linear model $\hat{y}_n = \mathbf{w}^\top x_n + b$.

(c) Application. You are asked to build an anomaly-detection system for industrial sensor logs where no anomaly labels exist. Which ML paradigm applies, and how should the data be represented?

5. Broader Understanding & Context

Formulating every problem as a tuple $(\mathcal{D}, \mathcal{M}, \mathcal{L}, \mathcal{A})$ of data, model, loss, and learning algorithm gives a universal blueprint. Every advance in later weeks (MLPs, CNNs, RNNs, Transformers) updates one or more elements of this tuple.

Task 2: Linear Regression, Design Matrix & Closed-Form Solutions

Covers Week 2 material

1. Prerequisites to Understanding

- Matrix multiplication, transposes, matrix derivatives, and matrix inverses.
- Rank of a matrix and when $\mathbf{X}^\top\mathbf{X}$ is invertible.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §1.1 (approx. pp. 4–11) and §3.1.1 (approx. pp. 138–143).
  Objective: Derive the sum-of-squares error function, construct the design matrix $\mathbf{X}$, and compute the closed-form weights from the normal equations.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 5 (approx. pp. 103–115).
  Objective: Implement parameter estimation with PyTorch tensor linear algebra (`torch.linalg.pinv`, `torch.linalg.lstsq`) and contrast it with iterative learning.

3. Practice Coding Assignment

Generate 50 noisy 1D sine-wave points ($y = \sin(2\pi x) + \epsilon$). Construct a degree-$M$ polynomial design matrix $\mathbf{X} \in \mathbb{R}^{N\times(M+1)}$ with $X_{i,j} = x_i^j$ using `torch.vander` (hint: pass `increasing=True`). Compute the weight vector $\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{t}$ directly, then again with `torch.linalg.lstsq`, and compare the two results.

Reading for this assignment: DLwPT Ch. 5: "Learning is just parameter estimation" (the book fits a linear model by gradient descent; compare with your closed-form answer). DLwPT Ch. 3: "The tensor API" for `matmul` and transposes. `torch.vander` and `torch.linalg.lstsq` are in the PyTorch docs rather than the book.

4. Practice Test Problems

(a) General Understanding. Why are polynomial curve-fitting models classified as *linear regression* models even though they predict non-linear curves?

(b) Math Needed. Expand the loss $E(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{t}\|_2^2$ into matrix terms, compute $\nabla_{\mathbf{w}} E(\mathbf{w})$, and derive the normal equations and $\mathbf{w}^*$.

(c) Application. A dataset has $D = 100{,}000$ features. Why is computing $\mathbf{w}^*$ with the closed-form normal equations computationally infeasible?

5. Broader Understanding & Context

Closed-form solutions give exact baselines for simple linear problems. Recognizing their $O(D^3)$ ceiling explains why iterative optimization (gradient descent) is mandatory at deep-learning scale.

Task 3: Probabilistic ML & Maximum Likelihood

Covers Week 2 material

1. Prerequisites to Understanding

- Probability density functions, Gaussian distributions, and logarithm rules ($\log ab = \log a + \log b$).
- Independence and the product rule for likelihoods.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §1.2.5 (approx. pp. 23–29) and §3.1.1 (approx. pp. 140–141).
  Objective: Derive how Gaussian additive noise turns maximum likelihood estimation (MLE) into mean-squared-error minimization.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 5 (approx. pp. 115–125).
  Objective: Program a custom negative log-likelihood loss function in PyTorch.

3. Practice Coding Assignment

Implement `gaussian_nll_loss(y_pred, y_true, sigma)` that returns the mean over the batch of $-\log \mathcal{N}(y \mid \hat{y}, \sigma^2)$. For fixed σ, verify numerically (with `autograd`) that the gradient of your loss with respect to `y_pred` equals $\frac{1}{2\sigma^2}$ times the gradient of `nn.MSELoss()` (mean reduction). Then check that minimizing either loss yields the same fitted parameters.

Reading for this assignment: DLwPT Ch. 5: the loss-function discussion inside "Learning is just parameter estimation". DLwPT Ch. 6: "The PyTorch nn module" (`nn.MSELoss`).

4. Practice Test Problems

(a) General Understanding. What is the fundamental statistical justification for using the mean squared error objective in regression?

(b) Math Needed. Prove that minimizing the negative log-likelihood under i.i.d. Gaussian noise yields the sum-of-squares (MSE) objective.

(c) Application. How can a model predict both the mean target value and the data uncertainty (heteroscedastic noise) for each input $x^{(i)}$?

5. Broader Understanding & Context

Connecting probability to optimization shows that loss functions are not arbitrary heuristics. Reading every objective as a negative log-likelihood gives a clean probabilistic framework that extends to classification (cross-entropy).

Task 4: Linear Classification, Softmax & Cross-Entropy

Covers Week 3 material

1. Prerequisites to Understanding

- Bernoulli and categorical distributions, softmax normalization.
- Basic information theory: entropy and expectation of $-\log p$.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §4.2 (approx. pp. 192–197), §4.3.4 (approx. pp. 206–209), and §1.6 Information Theory (approx. pp. 48–58).
  Objective: Understand the softmax mapping, entropy $H(p)$, cross-entropy $H(p, q)$, and KL divergence $\mathrm{KL}(p \parallel q)$.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 6 (approx. pp. 143–150) and Chapter 7 (approx. pp. 173–187).
  Objective: Use `nn.CrossEntropyLoss` correctly: it takes raw logits, not probabilities, for numerical stability.

3. Practice Coding Assignment

1. Write a numerically stable softmax in Python using the shift $z_i' = z_i - \max(\mathbf{z})$.
2. Compute `nn.CrossEntropyLoss()` for logits $\mathbf{z} = [1002.0, 1001.0, 1000.0]$ and true label $k = 0$. Then compute softmax *without* the max subtraction in float32 and observe the overflow (`inf`/`nan`).

Reading for this assignment: DLwPT Ch. 7 ("Telling birds from airplanes"): the sections on representing the output as probabilities (softmax) and on the classification loss (`nn.LogSoftmax` + `nn.NLLLoss`; `nn.CrossEntropyLoss` combines the two).

4. Practice Test Problems

(a) General Understanding. Why is cross-entropy preferred over mean squared error when training classifiers with softmax outputs?

(b) Math Needed. A 3-class problem has raw logits $\mathbf{z} = [2.0, 1.0, 0.1]$ and true class index $k = 0$ (one-hot target $\mathbf{p} = [1, 0, 0]$). (i) Compute the softmax probability vector $\mathbf{q}$. (ii) Compute the cross-entropy loss $L_{\mathrm{CE}} = -\log q_0$.

(c) Application. What does minimizing cross-entropy mean in information-theoretic terms?

5. Broader Understanding & Context

Softmax plus cross-entropy is the standard classification engine across modern neural networks. The information-theoretic view explains how a loss function shapes the model's beliefs and decision boundaries.

Task 5: Multilayer Perceptrons & Activation Functions

Covers Week 4 material

1. Prerequisites to Understanding

- Linear transformations and matrix-multiplication dimensions.
- Vector calculus (derivatives of elementwise functions).

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §5.1 (approx. pp. 225–231).
  Objective: Understand layer stacking, hidden units, and non-linear activations (sigmoid, tanh, ReLU).
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 6 (approx. pp. 151–163).
  Objective: Build multi-layer networks with `nn.Sequential` and `nn.Module`.

3. Practice Coding Assignment

Implement a 3-layer MLP with `nn.Module`:

$$X \in \mathbb{R}^{B\times 784} \to \mathrm{FC}(784, 256) \to \mathrm{ReLU} \to \mathrm{FC}(256, 64) \to \mathrm{GELU} \to \mathrm{FC}(64, 10).$$

Pass a random tensor through the model and verify the output shape is $\mathbb{R}^{B\times 10}$. Also count the trainable parameters with `sum(p.numel() for p in model.parameters())`.

Reading for this assignment: DLwPT Ch. 6: "Artificial neurons" (activation functions) and "The PyTorch nn module"; DLwPT Ch. 7: the fully connected model section (`nn.Sequential`, subclassing `nn.Module`).

4. Practice Test Problems

(a) General Understanding. What happens if you stack 5 fully connected layers without non-linear activations between them?

(b) Math Needed. Given $\mathbf{x} = [1,\, 2,\, -1]^\top$, $W_1 = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \end{bmatrix}$, $\mathbf{b}_1 = [0,\, 1]^\top$, a ReLU hidden layer, $W_2 = [1 \;\; 2]$, and $b_2 = -1$, compute the pre-activation $\mathbf{z}$, the activation $\mathbf{h}$, and the scalar output $\hat{y}$.

(c) Application. Why has ReLU ($\max(0, z)$) largely replaced sigmoid activations in the hidden layers of deep networks?

5. Broader Understanding & Context

MLPs introduce representation learning through hidden layers. This architecture is the building block underneath CNNs, RNNs, and Transformers.

Task 6: Backpropagation, Initialization & Regularization

Covers Week 4 material

1. Prerequisites to Understanding

- Multivariate chain rule.
- Variance properties of sums and products of independent random variables.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §5.3 (approx. pp. 241–249) and §5.5 (approx. pp. 256–261).
  Objective: Trace backpropagation chain-rule mechanics and derive L_2 weight decay. (Xavier/Glorot initialization and dropout post-date PRML: see Glorot & Bengio, 2010 and Srivastava et al., 2014, or your lecture slides.)
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 5 (approx. pp. 126–135) and the regularization subsection of Chapter 8 (weight penalties, dropout).
  Objective: Train with weight decay (`optim.SGD(..., weight_decay=1e-4)`) and `nn.Dropout`.

3. Practice Coding Assignment

1. Implement one manual backpropagation step in NumPy for a 1-hidden-layer network (suggested: tanh hidden layer, squared-error loss).
2. Verify your gradients against `loss.backward()` and `w.grad` in PyTorch using the same weights and data.

Reading for this assignment: DLwPT Ch. 5: "PyTorch's autograd: backpropagating all things" (`requires_grad`, `.backward()`, `.grad`, zeroing gradients) and the optimizers section. DLwPT Ch. 8: "Helping our model to converge and generalize" (weight penalties, dropout).

4. Practice Test Problems

(a) General Understanding. What problem does Xavier (Glorot) initialization solve, and how does it solve it?

(b) Math Needed. In inverted dropout with drop probability p, a hidden unit h becomes h' during training, where

$$h' = \begin{cases} 0 & \text{with probability } p, \\ \dfrac{h}{1-p} & \text{with probability } 1-p. \end{cases}$$

Prove $\mathbb{E}[h'] = h$ and explain why no rescaling is needed at test time.

(c) Application. A model overfits (low training loss, high validation loss). List 4 distinct regularization techniques that can improve generalization.

5. Broader Understanding & Context

Backpropagation makes efficient optimization of deep networks possible. Sound initialization and regularization keep gradients healthy and prevent overfitting, which together make stable deep learning practical.

Task 7: Convolutional Neural Network Fundamentals

Covers Week 5 material

1. Prerequisites to Understanding

- 2D grids, discrete cross-correlation/convolution.
- Tensor shape conventions: $N \times C \times H \times W$.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML):** §5.5.6 (approx. pp. 267–269).
  Objective: Master the spatial inductive biases of CNNs: locality, translation invariance, and weight sharing.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 8 (approx. pp. 193–207).
  Objective: Build spatial feature extractors with `nn.Conv2d` and `nn.MaxPool2d`.

3. Practice Coding Assignment

Given an RGB image tensor $X \in \mathbb{R}^{1\times3\times32\times32}$:

1. Pass it through `nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)`.
2. Pass the result through `nn.MaxPool2d(kernel_size=2, stride=2)`.

Print the tensor shape after each stage and the parameter count of the convolution.

Reading for this assignment: DLwPT Ch. 8: "The case for convolutions", "Convolutions in action" (kernel size, padding), "Subclassing nn.Module", and the down-sampling discussion (`nn.MaxPool2d`).

4. Practice Test Problems

(a) General Understanding. Compare fully connected (FC) layers with convolutional layers in terms of parameter counts and preservation of spatial structure.

(b) Math Needed. State the general output-size formula for a 2D convolution with input size H_{in}, padding P, kernel size K, and stride S. Then compute the output size for $H_{in} = 55$, $P = 2$, $K = 5$, $S = 2$.

(c) Application. An input feature map has shape $C_{in} \times H \times W = 64 \times 32 \times 32$. A convolution layer has $C_{out} = 128$ filters, kernel size $K = 3$, padding $P = 1$, stride $S = 1$. How many learnable parameters (including biases) does the layer have?

5. Broader Understanding & Context

CNNs bake in spatial inductive biases suited to grid data. Locality and weight sharing let vision models scale without the parameter explosion of dense layers.

Task 8: Modern CNN Architectures & Spatial Operations

Covers Week 6 material

1. Prerequisites to Understanding

- Residual connections and the idea of an identity mapping.
- Channel-wise operations on feature maps; convolution as a (sparse) matrix multiplication.

2. Textbook Readings & Learning Objectives

- **Course lecture notes:** Slide Deck 05 (CNNs: modern architectures); supplementary: He et al., 2016 (ResNet), Lin et al., 2013 (Network in Network), Tolstikhin et al., 2021 (MLP-Mixer).
  Objective: Understand 1×1 convolutions (channel mixing), ResNet skip connections, transposed convolutions (up-sampling), and MLP-Mixer.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 8 (approx. pp. 208–229).
  Objective: Construct custom residual blocks as `nn.Module` subclasses with identity skip connections.

3. Practice Coding Assignment

Implement a ResNet basic block in PyTorch:

$$F(x) = \text{Conv2d} \to \text{BatchNorm} \to \text{ReLU} \to \text{Conv2d} \to \text{BatchNorm},$$

$$\text{Output} = \text{ReLU}(F(x) + x).$$

Add a 1×1 projection convolution on the identity path when the input and output channel counts (or spatial size, via stride) differ. Test with an input of shape `[2, 64, 32, 32]` for both a 64→64 block and a 64→128 stride-2 block.

Reading for this assignment: DLwPT Ch. 8: the model-design section on going deeper (skip connections and ResNet-style blocks, batch norm). 1×1 convolutions and `nn.ConvTranspose2d` are covered in the PyTorch docs rather than the book.

4. Practice Test Problems

(a) General Understanding. How do residual skip connections ($y = F(x) + x$) prevent degradation in very deep networks?

(b) Math Needed. (i) What is the function of a 1×1 convolution on a tensor of shape $C_{in} \times H \times W$? Write the output at spatial location (i, j). (ii) Compute the number of parameters (with bias) of a 1×1 convolution and of a 3×3 convolution that each map 256 channels to 64 channels, and give their ratio.

(c) Application. In generative or dense-prediction tasks (e.g. super-resolution, segmentation), how do transposed convolutions up-sample low-resolution feature maps? Include the output-size formula for kernel K, stride S, and padding P.

5. Broader Understanding & Context

1×1 convolutions and residual connections solved the degradation problem and made very deep networks trainable. These ideas carried into modern backbones, including Vision Transformers and MLP-Mixers.

Task 9: Recurrent Neural Networks & Sequential Data

Covers Week 7 material

1. Prerequisites to Understanding

- Time-series/sequence processing.
- Conditional-probability chain rule and joint sequence distributions.

2. Textbook Readings & Learning Objectives

- **Bishop (PRML) and course slides:** §13.1 (approx. pp. 605–610) and Slide Deck 06 (RNNs).
  Objective: Formulate sequence probabilities $P(x_1, \ldots, x_T)$, compare Markov autoregressive and latent-variable models, and apply the stationarity assumption.
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 4 (approx. pp. 87–93).
  Objective: Process sequential data tensors and build recurrent unrolling loops (compare with `nn.RNNCell/nn.RNN` from the PyTorch docs).

3. Practice Coding Assignment

Write a bare-metal character-level RNN step loop in PyTorch using raw tensor parameters (no `nn.RNN`):

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad \hat{y}_t = W_{hy}h_t + b_y.$$

Process an input sequence of length $T = 10$ and print the output logits at every time step.

Reading for this assignment: DLwPT Ch. 4: "Working with time series" and "Representing text" (one-hot encoding, embeddings) for sequence tensor layouts. The book does not cover `nn.RNN`; use the PyTorch documentation for `nn.RNNCell/nn.RNN`.

4. Practice Test Problems

(a) General Understanding. Explain the **stationarity assumption** in time-series modeling and why it is necessary.

(b) Math Needed. Given the chain-rule factorization $P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{t-1}, \ldots, x_1)$, contrast an n-gram autoregressive model ($\tau = n - 1$) with a latent autoregressive (RNN) model in terms of how history is represented and how parameters scale.

(c) Application. What structural weakness makes standard RNNs struggle to learn long-term dependencies over long sequences ($T > 100$)?

5. Broader Understanding & Context

RNNs introduced continuous hidden states for variable-length sequential modeling. Their long-range gradient limits motivate parallel, non-recurrent sequence architectures (Transformers).

Task 10: Attention, Transformers & Self-Supervised BERT

Covers Week 8 material

1. Prerequisites to Understanding

- Inner products as similarity measures.
- Softmax-weighted averages and self-supervised pre-training ideas.

2. Textbook Readings & Learning Objectives

- **Course lecture notes:** Slide Deck 07 (Attention and Transformers); supplementary: Vaswani et al., 2017 ("Attention Is All You Need"), Devlin et al., 2019 (BERT).
  Objective: Master query/key/value dot-product attention, multi-head attention, and the BERT pre-training objectives (masked language modeling and next-sentence prediction).
- **Stevens, Antiga, Viehmann (DLwPT):** Chapter 4 (approx. pp. 93–101).
  Objective: Build scaled dot-product attention from PyTorch matrix operations (`torch.matmul`, `torch.softmax`). The book predates Transformers, so use it for tensor/text representation only.

3. Practice Coding Assignment

Implement scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Given $X \in \mathbb{R}^{B\times T\times d_{\text{model}}}$, compute $Q = XW_Q$, $K = XW_K$, $V = XW_V$, form the $T \times T$ attention matrix, and output the context tensor $Z \in \mathbb{R}^{B\times T\times d_v}$. Verify that each row of the attention matrix sums to 1.

Reading for this assignment: DLwPT Ch. 4: "Representing text" (one-hot, embeddings) as the input to attention; DLwPT Ch. 3 (`matmul`, transposes) and Ch. 7 (softmax). Attention itself is in the slides and the PyTorch docs.

4. Practice Test Problems

(a) General Understanding. Describe the two self-supervised pre-training objectives used to train BERT without human-annotated labels.

(b) Math Needed. Let $\mathbf{q} \in \mathbb{R}^{d_k}$ be a query and $K \in \mathbb{R}^{T\times d_k}$ the key matrix. (i) Why is $\mathbf{q}K^\top$ divided by $\sqrt{d_k}$ before the softmax? (ii) Prove the effect of large d_k on the variance of the dot product.

(c) Application. How does multi-head attention increase representational capacity compared with single-head attention?

5. Broader Understanding & Context

Attention replaces the sequential bottleneck of recurrence with parallel matrix computation. Self-supervised pre-training lets large models learn contextual representations from unlabeled text, the foundation of modern large language models.